

Lab 5

Inheritance

The capability of a class to derive properties and characteristics from another class is called **Inheritance**. Inheritance is one of the most important features of Object-Oriented Programming in C++.

Syntax

```
class DerivedClass : mode_of_inheritance BaseClass {  
    // Body of the Derived Class  
};
```

where **mode of inheritance** controls the access level of the inherited members of the base class in the derived class.

Mode	Description
Public Inheritance Mode	Public member of the base class will become public in the derived class and protected members of the base class will become protected in the derived class.
Protected Inheritance Mode	Both public and protected members of the base class will become protected in the derived class.
Private Inheritance Mode	Both public members and protected members of the base class will become private in the derived class. Private mode is the default mode that is applied when we don't specify any mode.

Example 1:

```
using namespace std;

class Base {
public:
    int n;
    void printN() {
        cout << n << endl;
    }
};

// Inheriting Base class publicly
class Derived : public Base {
public:
    void func () {

        // Accessing Base class members
        n = 22;
    }
};

int main() {

    // Creating objects of derived
    Derived d;

    // Accessing Derived class member
    d.func();

    // Accessing Base class member
    d.printN();
    return 0;
}
```

Output:

```
22
```

Example 2:

```
using namespace std;

class Vehicle {
public:
    Vehicle() {
        cout << "This is a Vehicle"<< endl;
    }
};

// Sub class derived from a single base classes
class Car : public Vehicle {
public:
    Car() {
        cout << "This Vehicle is Car"<< endl;
    }
};

int main() {

    // Creating object of sub class will
    // invoke the constructor of base classes
    Car obj;
    return 0;
}
```

Output

```
This is a Vehicle
This Vehicle is Car
```

Example 3:

```
using namespace std;

// Base class that is to be inherited
class Parent {
public:
    int id_p;
    Parent(int x = 22) : id_p(x) {}
    void printID_p() {
        cout << "Base ID: " << id_p << endl;
    }
};

// Derived publicly inheriting from Base
// Class
class Child : public Parent {
public:
    int id_c;
    Child(int x = 22) : id_c(x) {}
    void printID_c() {
        cout << "Child ID: " << id_c << endl;
    }
};

int main() {
    Child obj1;

    // An object of class child has all data members
    // and member functions of class parent
    // so we try accessing the parents method and data from
    // the child class object.
    obj1.id_p = 7;
    obj1.printID_p();

    // finally accessing the child class methods and data
    // too
    obj1.id_c = 91;
    obj1.printID_c();
    return 0;
}
```

Output

```
Base ID: 7
Child ID: 91
```

Example 4:

```
using namespace std;

class Vehicle {
public:
    Vehicle() {
        cout << "This is a Vehicle"<< endl;
    }
};

class fourWheeler : public Vehicle {
public:
    fourWheeler() {
        cout << "4 Wheeler Vehicles"<< endl;
    }
};

class Car : public fourWheeler {
public:
    Car() {
        cout << "This 4 Wheeler Vehicles is a Car";
    }
};

int main() {

    // Creating object of sub class will
    // invoke the constructor of base classes.
    Car obj;
    return 0;
}
```

Output

This is a Vehicle

4 Wheeler Vehicles

This 4 Wheeler Vehicles is a Car